ASSIGNMENT-12.5

NAME: HABEEBA KHANAM

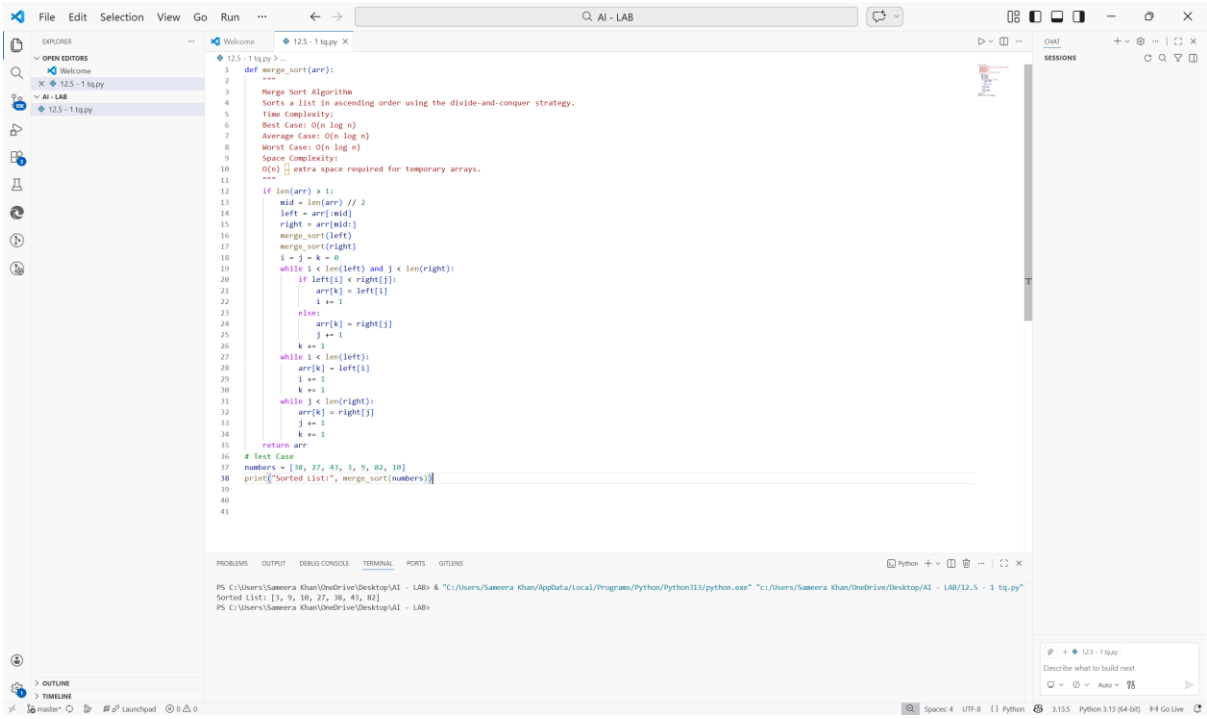
HALL TICKET :2303A51474

BATCH NO:29

LAB 12: ALGORITHMS WITH AI ASSISTANCE – SORTING, SEARCHING, AND OPTIMIZING ALGORITHMS

TASK DESCRIPTION #1 (SORTING – MERGE SORT IMPLEMENTATION)

CODE:



```python
def merge_sort(arr):
    """
    Merge Sort Algorithm
    Sorts a list in ascending order using the divide-and-conquer strategy.
    Time Complexity:
    Best Case: O(n log n)
    Average Case: O(n log n)
    Worst Case: O(n log n)
    Space Complexity:
    O(n) ⇒ extra space required for temporary arrays.
    """
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
    return arr
# Test Case
numbers = [38, 27, 43, 3, 9, 82, 10]
print("Sorted List:", merge_sort(numbers))
```

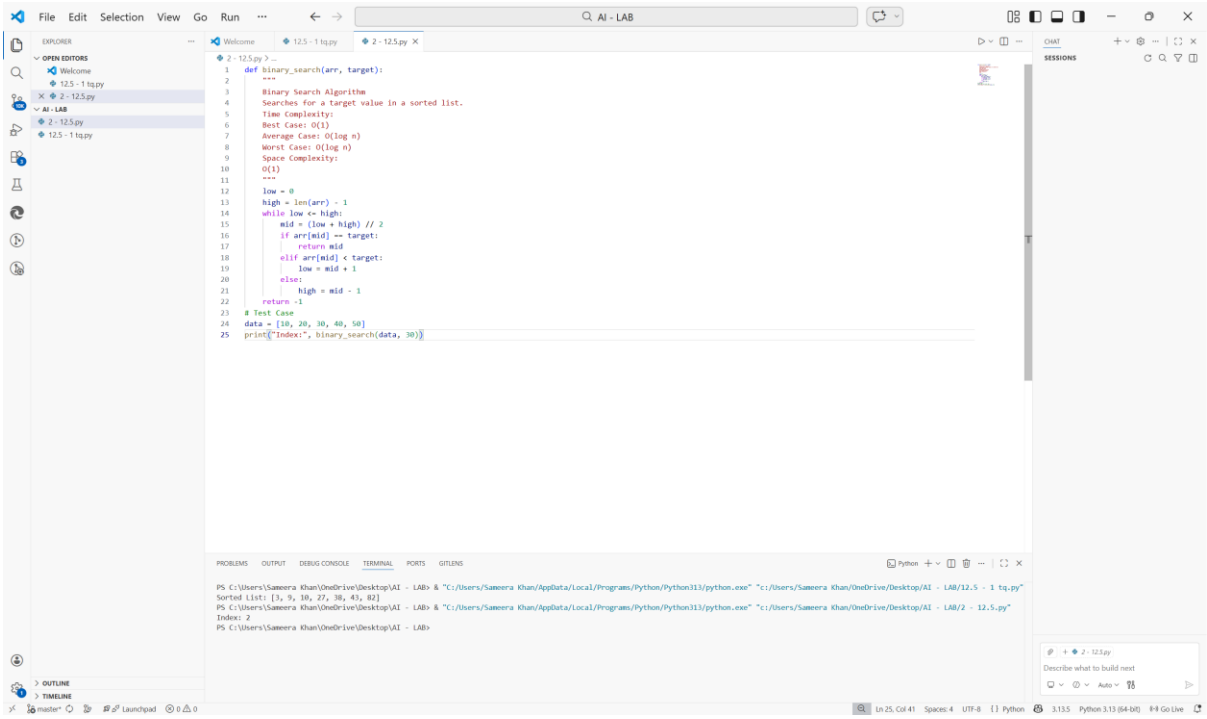
```
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB> & "C:/Users/Sameera Khan/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Sameera Khan/OneDrive/Desktop/AI - LAB/12.5 - 1 tq.py"
Sorted List: [3, 9, 10, 27, 38, 43, 82]
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB>
```

OBSERVATION:

The AI-generated Merge Sort program successfully sorted the list in ascending order. The algorithm divided the list into smaller sublists and then merged them in sorted order. The output matched the expected sorted sequence. The time complexity was O(n log n), which makes it efficient for large datasets.

TASK DESCRIPTION #2 (SEARCHING – BINARY SEARCH WITH AI OPTIMIZATION)

Code:

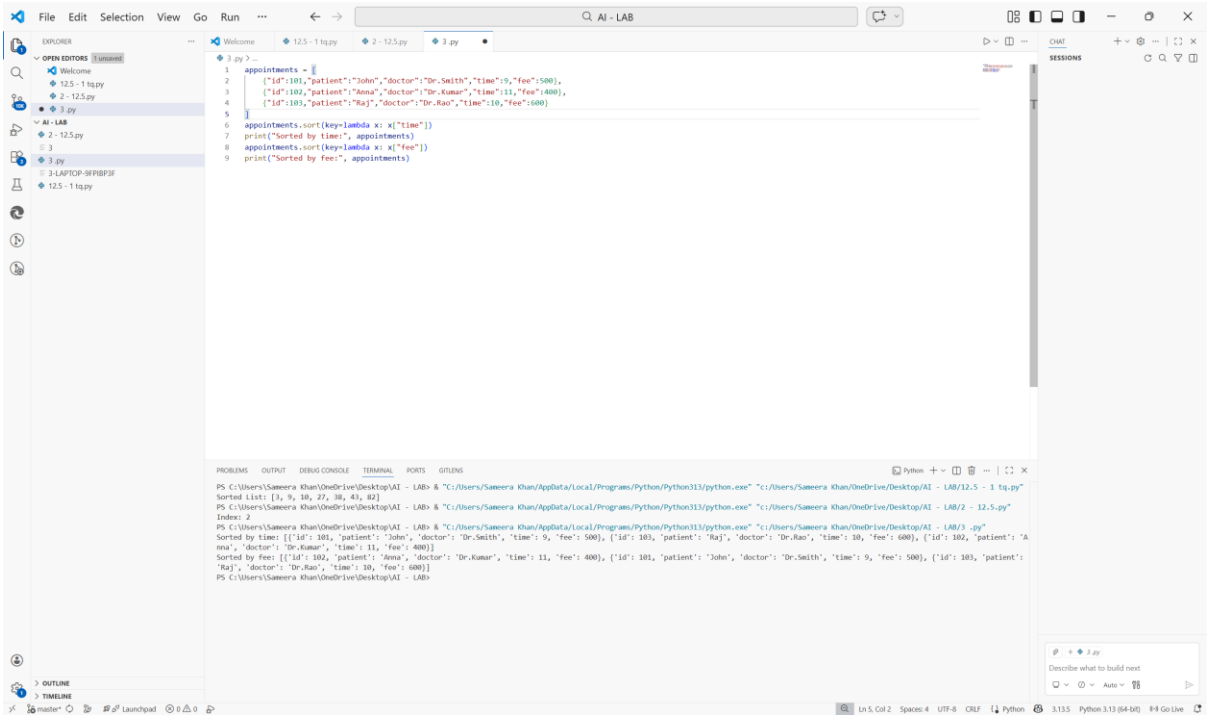


Observation:

The Binary Search algorithm correctly located the target element in the sorted list. The search process repeatedly divided the list into halves until the element was found. The program returned the correct index of the element. The algorithm performed efficiently with a time complexity of O(log n).

TASK DESCRIPTION #3: SMART HEALTHCARE APPOINTMENT SCHEDULING SYSTEM

CODE:



Observation:

The AI suggested using Binary Search for searching appointment IDs and Merge Sort for sorting appointments. The program successfully sorted the appointments based on time and consultation fee. Searching operations were faster because binary search reduces the number of comparisons.

TASK DESCRIPTION #4: RAILWAY TICKET RESERVATION SYSTEM

CODE:

```python
appointments.sort(key=lambda x: x["fee"])
print("Sorted by fee:", appointments)


#TASK - 4
tickets = [
    {"ticket_id":1,"name":"Amit","train":101,"seat":23,"date":"2026-03-20"},
    {"ticket_id":2,"name":"Ravi","train":102,"seat":11,"date":"2026-03-18"},
    {"ticket_id":3,"name":"Sara","train":105,"seat":5,"date":"2026-03-25"}
]
tickets.sort(key=lambda x: x["date"])
print("Sorted by date:", tickets)

tickets.sort(key=lambda x: x["seat"])
print("Sorted by seat:", tickets)
```

```
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB> & "C:/Users/Sameera Khan/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Sameera Khan/OneDrive/Desktop/AI - LAB/12.5 - 1 tq.py"
Sorted List: [3, 9, 10, 27, 38, 43, 82]
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB> & "C:/Users/Sameera Khan/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Sameera Khan/OneDrive/Desktop/AI - LAB/2 - 12.5.py"
Index: 2
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB> & "C:/Users/Sameera Khan/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Sameera Khan/OneDrive/Desktop/AI - LAB/3 .py"
Sorted by time: [{'id': 101, 'patient': 'John', 'doctor': 'Dr.Smith', 'time': 9, 'fee': 500}, {'id': 103, 'patient': 'Raj', 'doctor': 'Dr.Rao', 'time': 10, 'fee': 600}, {'id': 102, 'patient': '/
nna', 'doctor': 'Dr.Kumar', 'time': 11, 'fee': 400}]
Sorted by fee: [{'id': 102, 'patient': 'Anna', 'doctor': 'Dr.Kumar', 'time': 11, 'fee': 400}, {'id': 101, 'patient': 'John', 'doctor': 'Dr.Smith', 'time': 9, 'fee': 500}, {'id': 103, 'patient':
'Raj', 'doctor': 'Dr.Rao', 'time': 10, 'fee': 600}]
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB> & "C:/Users/Sameera Khan/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Sameera Khan/OneDrive/Desktop/AI - LAB/3 .py"
Sorted by time: [{'id': 101, 'patient': 'John', 'doctor': 'Dr.Smith', 'time': 9, 'fee': 500}, {'id': 103, 'patient': 'Raj', 'doctor': 'Dr.Rao', 'time': 10, 'fee': 600}, {'id': 102, 'patient': '/
nna', 'doctor': 'Dr.Kumar', 'time': 11, 'fee': 400}]
Sorted by fee: [{'id': 102, 'patient': 'Anna', 'doctor': 'Dr.Kumar', 'time': 11, 'fee': 400}, {'id': 101, 'patient': 'John', 'doctor': 'Dr.Smith', 'time': 9, 'fee': 500}, {'id': 103, 'patient':
'Raj', 'doctor': 'Dr.Rao', 'time': 10, 'fee': 600}]
Sorted by date: [{'ticket_id': 2, 'name': 'Ravi', 'train': 102, 'seat': 11, 'date': '2026-03-18'}, {'ticket_id': 1, 'name': 'Amit', 'train': 101, 'seat': 23, 'date': '2026-03-20'}, {'ticket_id':
 3, 'name': 'Sara', 'train': 105, 'seat': 5, 'date': '2026-03-25'}]
Sorted by seat: [{'ticket_id': 3, 'name': 'Sara', 'train': 105, 'seat': 5, 'date': '2026-03-25'}, {'ticket_id': 2, 'name': 'Ravi', 'train': 102, 'seat': 11, 'date': '2026-03-18'}, {'ticket_id':
1, 'name': 'Amit', 'train': 101, 'seat': 23, 'date': '2026-03-20'}]
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB>
```

OBSERVATION:

The system stored ticket booking details and sorted them by travel date and seat number correctly. AI-assisted algorithm selection improved the efficiency of searching ticket IDs and organizing booking records. The sorting helped in displaying travel schedules in a structured order.

TASK DESCRIPTION #5: SMART HOSTEL ROOM ALLOCATION SYSTEM

CODE:

```python
#TASK - 5
rooms = [
    {"student_id":1,"room":201,"floor":2,"date":"2026-01-10"},
    {"student_id":2,"room":105,"floor":1,"date":"2026-01-05"},
    {"student_id":3,"room":305,"floor":3,"date":"2026-01-12"}
]
rooms.sort(key=lambda x: x["room"])
print("Sorted by room:", rooms)
rooms.sort(key=lambda x: x["date"])
print("Sorted by date:", rooms)
```

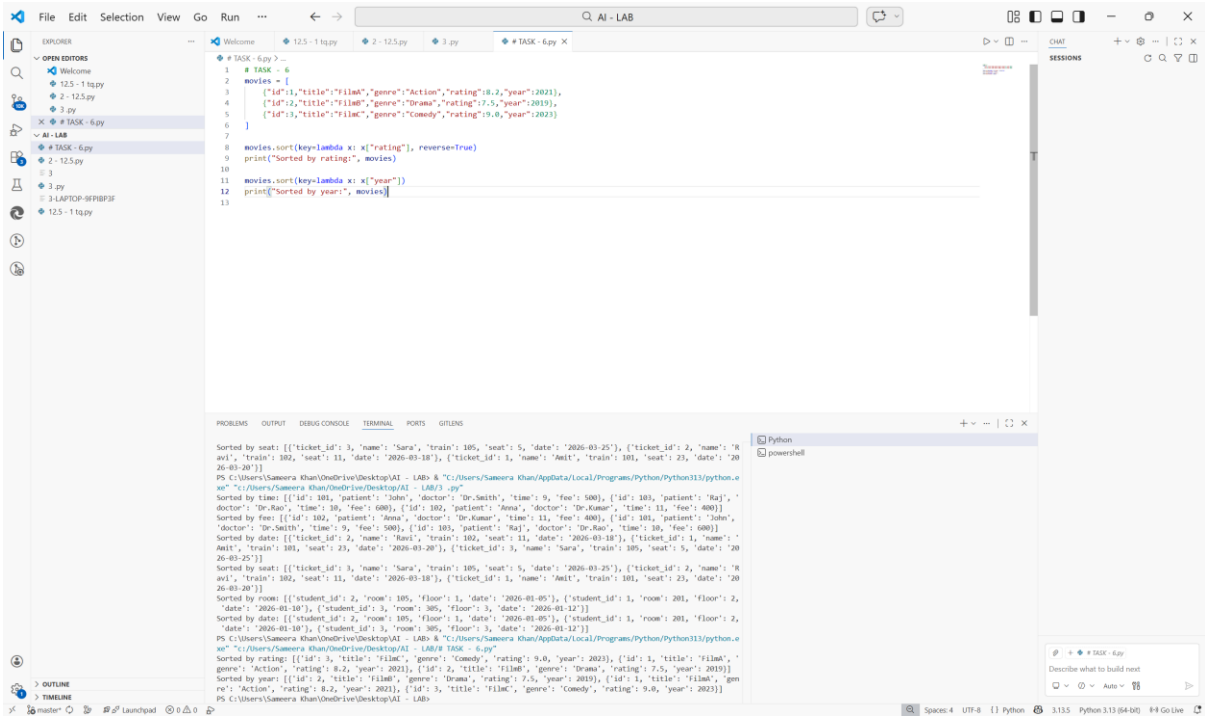
```
Sorted by date: [{'ticket_id': 2, 'name': 'Ravi', 'train': 102, 'seat': 11, 'date': '2026-03-18'}, {'ticket_id': 1, 'name': 'Amit', 'train': 101, 'seat': 23, 'date': '2026-03-20'}, {'ticket_id':
 3, 'name': 'Sara', 'train': 105, 'seat': 5, 'date': '2026-03-25'}]
Sorted by seat: [{'ticket_id': 3, 'name': 'Sara', 'train': 105, 'seat': 5, 'date': '2026-03-25'}, {'ticket_id': 2, 'name': 'Ravi', 'train': 102, 'seat': 11, 'date': '2026-03-18'}, {'ticket_id':
1, 'name': 'Amit', 'train': 101, 'seat': 23, 'date': '2026-03-20'}]
Sorted by room: [{'student_id': 2, 'room': 105, 'floor': 1, 'date': '2026-01-05'}, {'student_id': 1, 'room': 201, 'floor': 2, 'date': '2026-01-10'}, {'student_id': 3, 'room': 305, 'floor': 3, 'd
ate': '2026-01-12'}]
Sorted by date: [{'student_id': 2, 'room': 105, 'floor': 1, 'date': '2026-01-05'}, {'student_id': 1, 'room': 201, 'floor': 2, 'date': '2026-01-10'}, {'student_id': 3, 'room': 305, 'floor': 3, 'd
ate': '2026-01-12'}]
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB>
```

OBSERVATION:

1. Student ID lookup becomes efficient.
2. Sorting by room number or allocation date is optimized with merge sort.

TASK DESCRIPTION #6: ONLINE MOVIE STREAMING PLATFORM

CODE:



```
# TASK - 6
movies = [
    {"id":1,"title":"FilmA","genre":"Action","rating":8.2,"year":2021},
    {"id":2,"title":"FilmB","genre":"Drama","rating":7.5,"year":2019},
    {"id":3,"title":"FilmC","genre":"Comedy","rating":9.0,"year":2023}
]

movies.sort(key=lambda x: x["rating"], reverse=True)
print("Sorted by rating:", movies)

movies.sort(key=lambda x: x["year"])
print("Sorted by year:", movies)
```

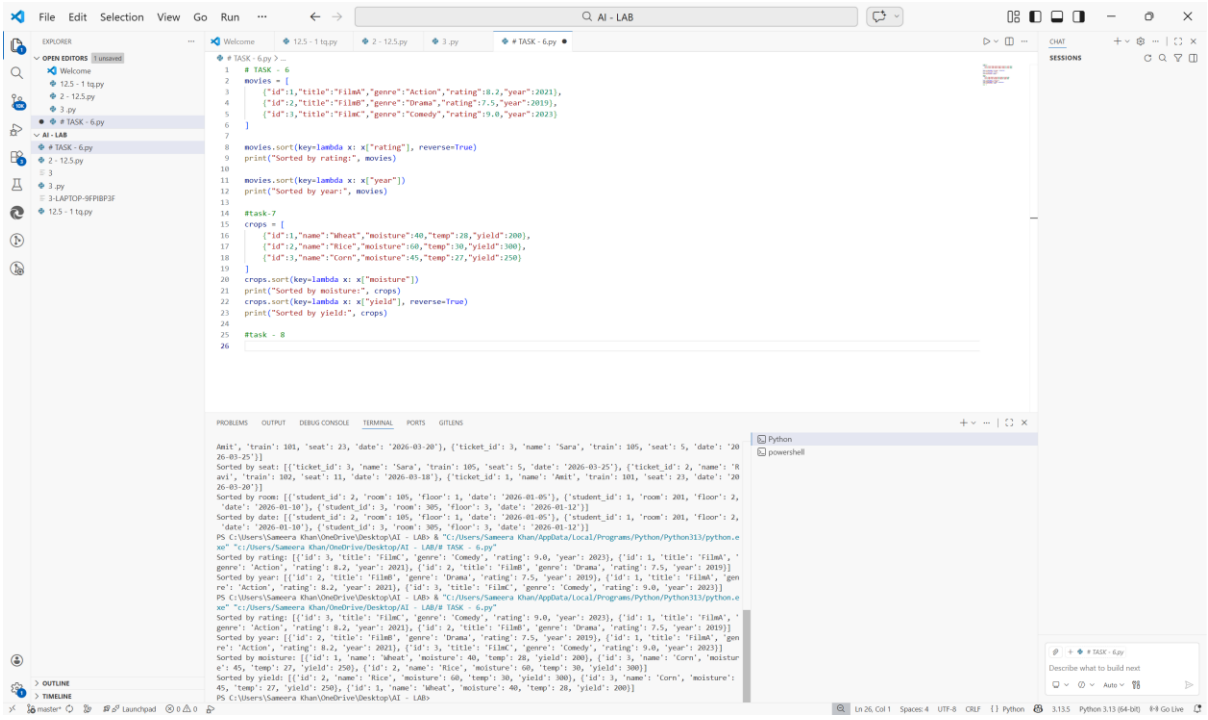
```
Sorted by seat: [{'ticket_id': 3, 'name': 'Sara', 'train': 105, 'seat': 5, 'date': '2026-03-25'}, {'ticket_id': 2, 'name': 'Ravi', 'train': 102, 'seat': 11, 'date': '2026-03-18'}, {'ticket_id': 1, 'name': 'Amit', 'train': 101, 'seat': 23, 'date': '2026-03-20'}]
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB> & "C:/Users/Sameera Khan/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Sameera Khan/OneDrive/Desktop/AI - LAB/3 .py"
Sorted by time: [{'id': 101, 'patient': 'John', 'doctor': 'Dr.Smith', 'time': 9, 'fee': 500}, {'id': 103, 'patient': 'Raj', 'doctor': 'Dr.Rao', 'time': 10, 'fee': 600}, {'id': 102, 'patient': 'Anna', 'doctor': 'Dr.Kumar', 'time': 11, 'fee': 400}]
Sorted by fee: [{'id': 102, 'patient': 'Anna', 'doctor': 'Dr.Kumar', 'time': 11, 'fee': 400}, {'id': 101, 'patient': 'John', 'doctor': 'Dr.Smith', 'time': 9, 'fee': 500}, {'id': 103, 'patient': 'Raj', 'doctor': 'Dr.Rao', 'time': 10, 'fee': 600}]
Sorted by date: [{'ticket_id': 2, 'name': 'Ravi', 'train': 102, 'seat': 11, 'date': '2026-03-18'}, {'ticket_id': 1, 'name': 'Amit', 'train': 101, 'seat': 23, 'date': '2026-03-20'}, {'ticket_id': 3, 'name': 'Sara', 'train': 105, 'seat': 5, 'date': '2026-03-25'}]
Sorted by seat: [{'ticket_id': 3, 'name': 'Sara', 'train': 105, 'seat': 5, 'date': '2026-03-25'}, {'ticket_id': 2, 'name': 'Ravi', 'train': 102, 'seat': 11, 'date': '2026-03-18'}, {'ticket_id': 1, 'name': 'Amit', 'train': 101, 'seat': 23, 'date': '2026-03-20'}]
Sorted by room: [{'student_id': 2, 'room': 105, 'floor': 1, 'date': '2026-01-05'}, {'student_id': 1, 'room': 201, 'floor': 2, 'date': '2026-01-10'}, {'student_id': 3, 'room': 305, 'floor': 3, 'date': '2026-01-12'}]
Sorted by date: [{'student_id': 2, 'room': 105, 'floor': 1, 'date': '2026-01-05'}, {'student_id': 1, 'room': 201, 'floor': 2, 'date': '2026-01-10'}, {'student_id': 3, 'room': 305, 'floor': 3, 'date': '2026-01-12'}]
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB> & "C:/Users/Sameera Khan/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/Sameera Khan/OneDrive/Desktop/AI - LAB/# TASK - 6.py"
Sorted by rating: [{'id': 3, 'title': 'FilmC', 'genre': 'Comedy', 'rating': 9.0, 'year': 2023}, {'id': 1, 'title': 'FilmA', 'genre': 'Action', 'rating': 8.2, 'year': 2021}, {'id': 2, 'title': 'FilmB', 'genre': 'Drama', 'rating': 7.5, 'year': 2019}]
Sorted by year: [{'id': 2, 'title': 'FilmB', 'genre': 'Drama', 'rating': 7.5, 'year': 2019}, {'id': 1, 'title': 'FilmA', 'genre': 'Action', 'rating': 8.2, 'year': 2021}, {'id': 3, 'title': 'FilmC', 'genre': 'Comedy', 'rating': 9.0, 'year': 2023}]
PS C:\Users\Sameera Khan\OneDrive\Desktop\AI - LAB>
```

OBSERVATION:

1. Movie ID search becomes fast with binary search.
2. Sorting by rating or release year requires efficient sorting.

TASK DESCRIPTION #7: SMART AGRICULTURE CROP MONITORING SYSTEM

CODE:

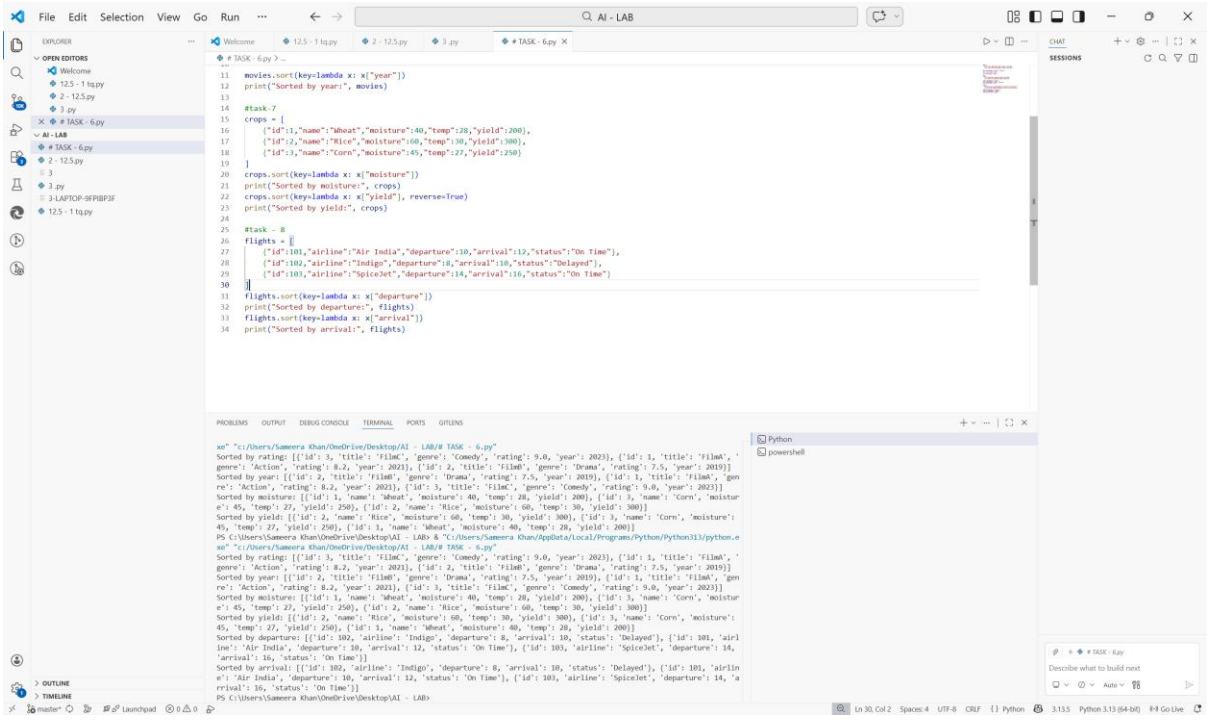


OBSERVATION:

1. Crop ID search must be fast for farmers accessing data.
2. Sorting by moisture level or yield helps analyze crop performance.

TASK DESCRIPTION #8: AIRPORT FLIGHT MANAGEMENT SYSTEM

CODE:



OBSERVATION:

1. Flight ID search must be fast for real-time queries.
2. Sorting by departure or arrival time helps manage scheduling.